**Hacker Women Council – Ethical Hacking**

**Actividad 04 - OWASP ZAP & WEB SECURITY CHALLENGE**

**Análisis de Seguridad Web con OWASP ZAP y Gin & Juice Shop**

**Alumno: Analía Rossotti**

**Fecha: Octubre de 2026**

**Primera evaluación de seguridad web con OWASP ZAP**

Aplicación evaluada: Gin & Juice Shop (https://ginandjuice.shop/)

En este trabajo documento una primera evaluación de seguridad web sobre Gin & Juice Shop, una aplicación deliberadamente vulnerable (el propio sitio lo indica en un cartel superior) pensada para practicar con escáneres de vulnerabilidades. Usé OWASP ZAP 2.17.0 como proxy entre el navegador y el sitio, y Google Chrome como navegador. Navegué el sitio de forma manual y ZAP analizó el tráfico que pasaba por el proxy, sin lanzar ataques activos.

El informe sigue las cinco partes de la guía: preparación del entorno, exploración de la aplicación, análisis de alertas, hallazgo principal y conclusiones.

## 1. Preparación del entorno

### 1.1 ZAP abierto

ZAP 2.17.0 en modo estándar con una sesión nueva. El árbol de Sitios todavía está vacío y el Historial no tiene peticiones: es el punto de partida limpio antes de navegar. En la barra de estado se ve que el proxy principal escucha en localhost:8080.

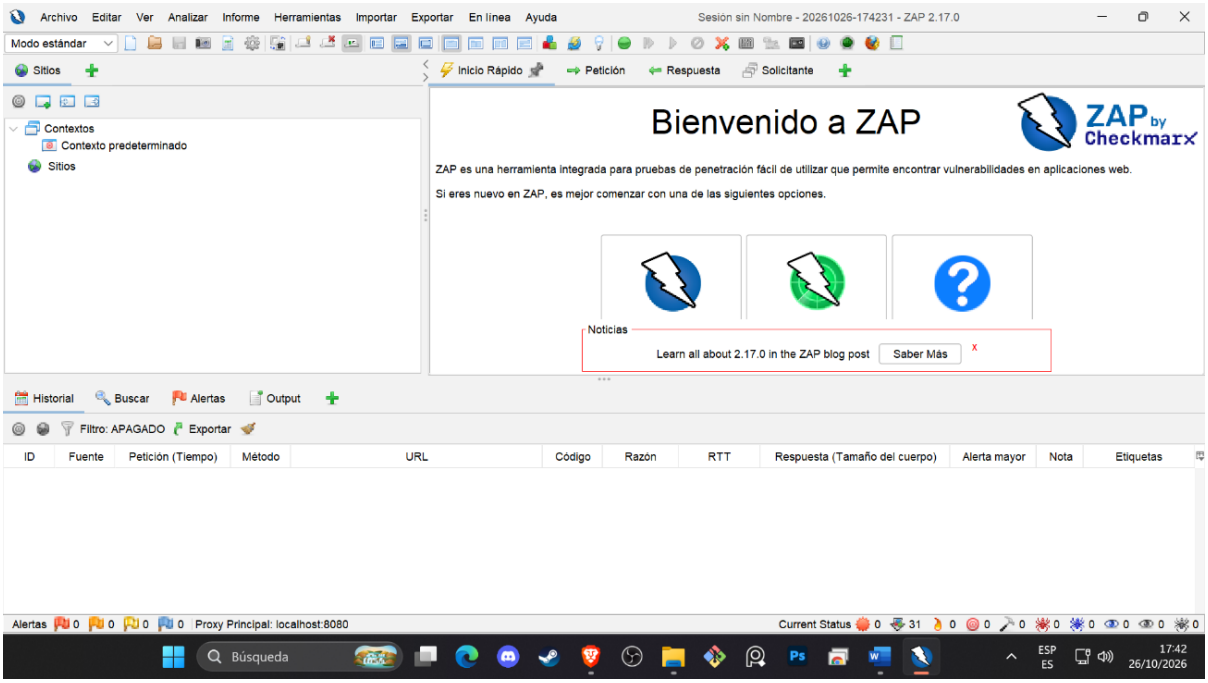



**Imagen 1.** *Interfaz principal de OWASP ZAP al iniciar, con el Historial vacío y el proxy en localhost:8080.*

### 1.2 Navegador configurado

Para que el tráfico pase por ZAP usé la opción Exploración Manual: cargué la URL a explorar (https://ginandjuice.shop/) y abrí Chrome con el botón Iniciar Navegador. De esta forma ZAP lanza el navegador ya configurado para usar su proxy (localhost:8080), sin tener que cargar a mano la dirección y el puerto en Chrome.

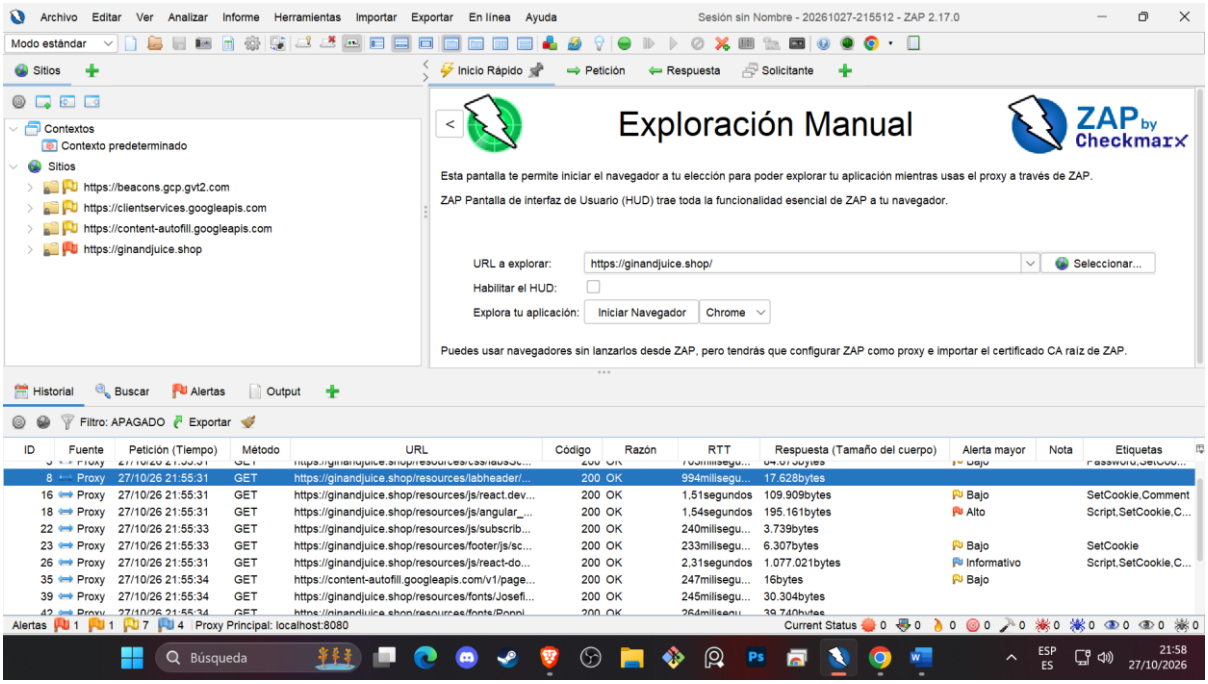



**Imagen 2.** *Pantalla de Exploración Manual de ZAP: URL a explorar, botón Iniciar Navegador (Chrome) y proxy principal localhost:8080 en la barra de estado.*

### 1.3 Acceso a https://ginandjuice.shop/

Con el navegador abierto ingresé a https://ginandjuice.shop/ y cargó la página principal (pestaña “Home - Gin & Juice Shop”). Chrome muestra “No es seguro” aunque la dirección use HTTPS: el tráfico pasa por el proxy de ZAP, que presenta su propio certificado, y el navegador no lo reconoce. Es un comportamiento esperable en este tipo de pruebas. Además, el cartel superior del sitio aclara que es una aplicación deliberadamente vulnerable para probar escáneres, por lo que analizarla está permitido.

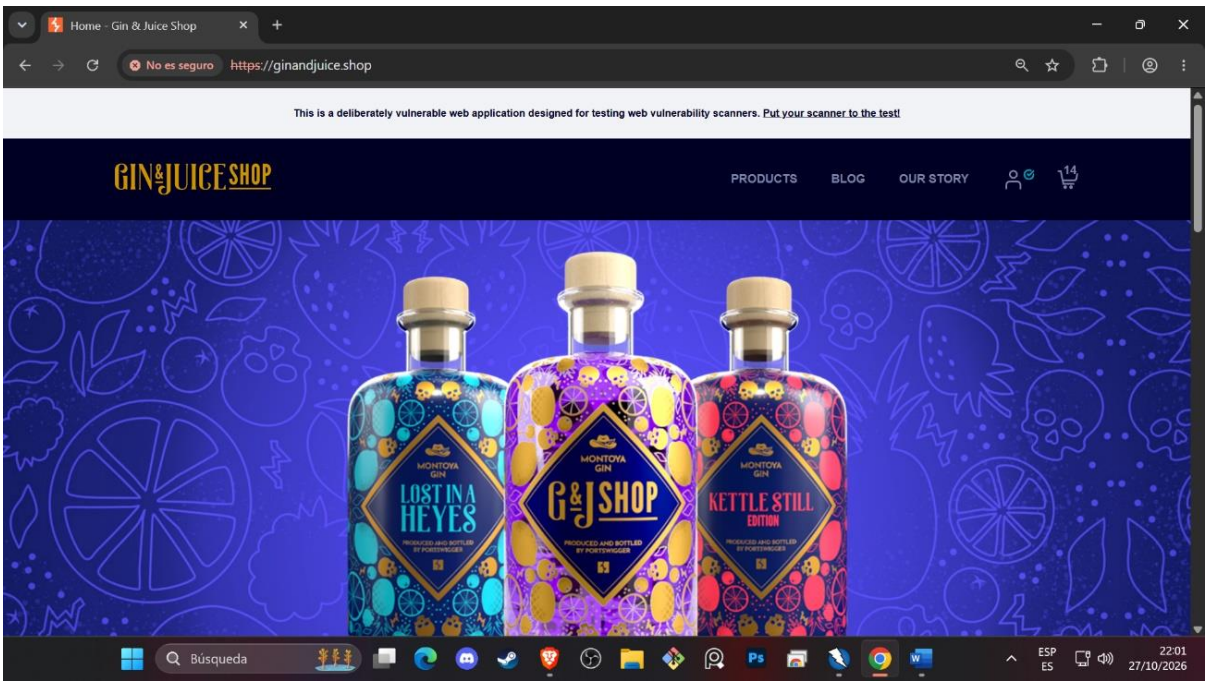



**Imagen 3.** *Acceso a https://ginandjuice.shop/ desde Chrome.*

### 1.4 Verificación de que el tráfico aparece en ZAP

En la pestaña Historial aparecen las peticiones generadas al cargar el sitio. La ID 1 es el GET a https://ginandjuice.shop/ con código 200 OK, seguida de pedidos a recursos (CSS, JavaScript, fuentes). Esto confirma que el tráfico del navegador pasa por ZAP. En el árbol de Sitios se ve el dominio ginandjuice.shop y también dominios de Google (googleapis.com, gvt2.com): son pedidos que Chrome hace en segundo plano y no forman parte de la aplicación evaluada.

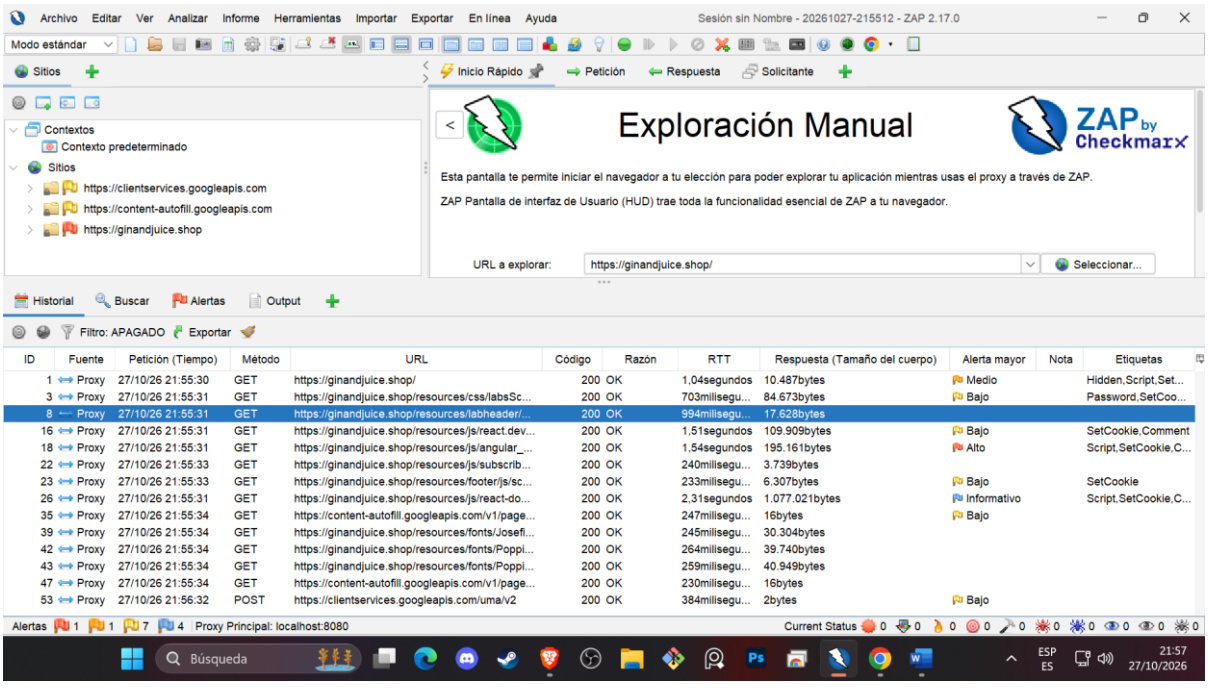



**Imagen 4.** *Historial de ZAP con las primeras peticiones a ginandjuice.shop y el árbol de Sitios.*

## 2. Exploración de la aplicación

### 2.1 Navegación en el sitio

Página principal de Gin & Juice Shop. La barra superior ofrece PRODUCTS, BLOG y OUR STORY, además de los íconos de usuario y carrito. Desde ahí recorrí las distintas secciones.

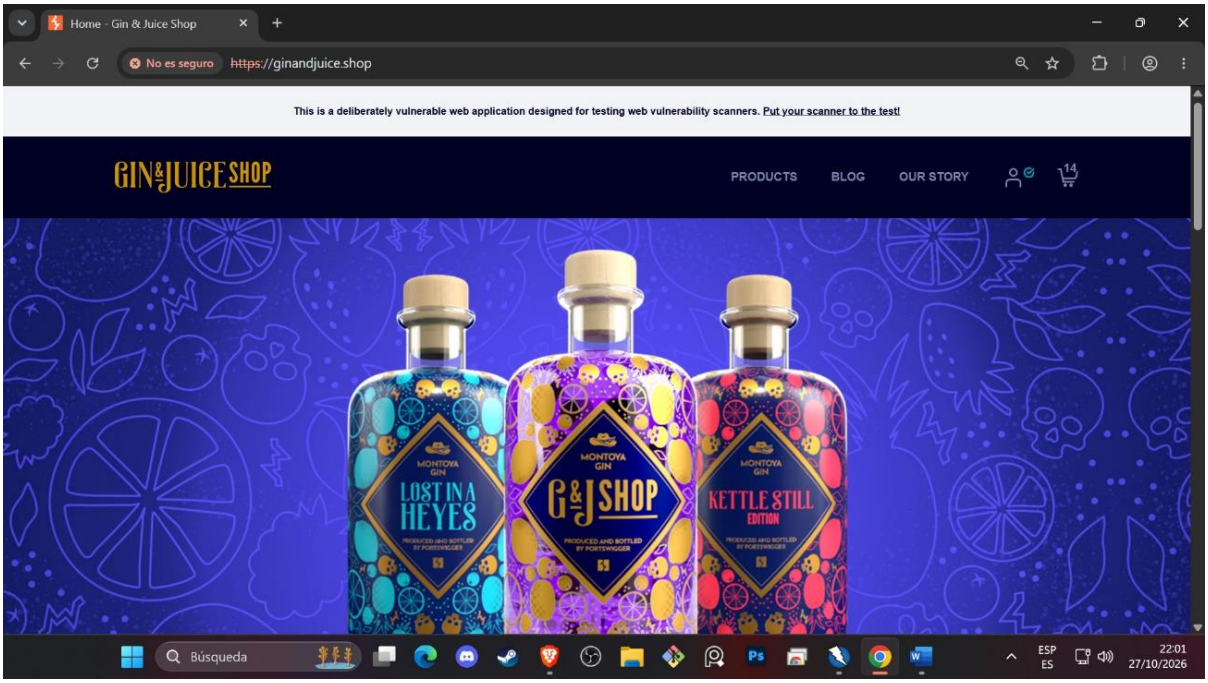



**Imagen 5.** *Página principal de Gin & Juice Shop.*

### 2.2 Tráfico HTTP en ZAP

Tráfico generado al navegar el sitio. Todas las peticiones a la aplicación son GET con código 200 OK. La columna “Alerta mayor” muestra el riesgo más alto que ZAP detectó en cada respuesta: por ejemplo, la ID 1 (página principal) tiene alerta Media, la ID 18 (un script JavaScript “angular_…”) tiene alerta Alta y la ID 26 (“react-do…”) es Informativa. La ID 53 es un POST, pero va a clientservices.googleapis.com, es decir, tráfico de Chrome y no de la aplicación.

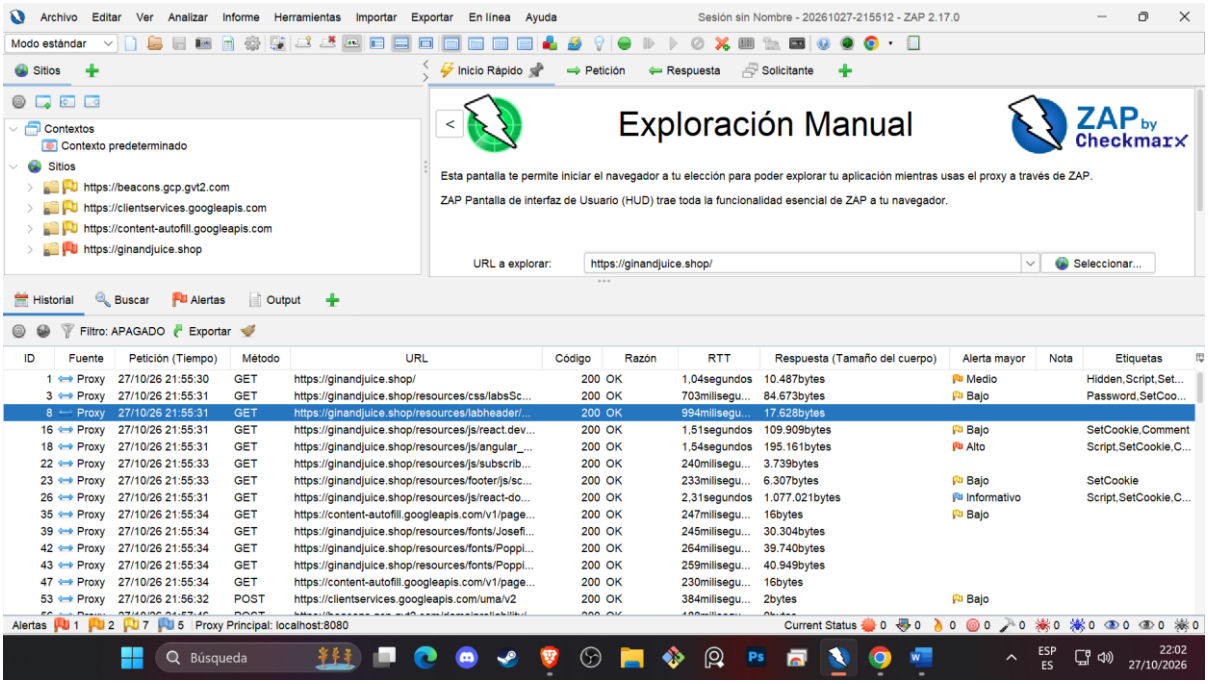



**Imagen 6.** *Historial de ZAP con el tráfico generado al navegar por el sitio (barra de estado: 1 Alta, 2 Medias, 7 Bajas, 5 Informativas).*

### 2.3 Sección de productos

Al entrar en Products se generaron pedidos a /catalog (ID 61) y al detalle de un producto, /catalog/product?prod… (ID 86); ambos tienen alerta Media. También se ve la carga de scripts propios de la página de producto (xmlStock…, stockChe…, que por su nombre parecen relacionados con la consulta de stock) y de fuentes.

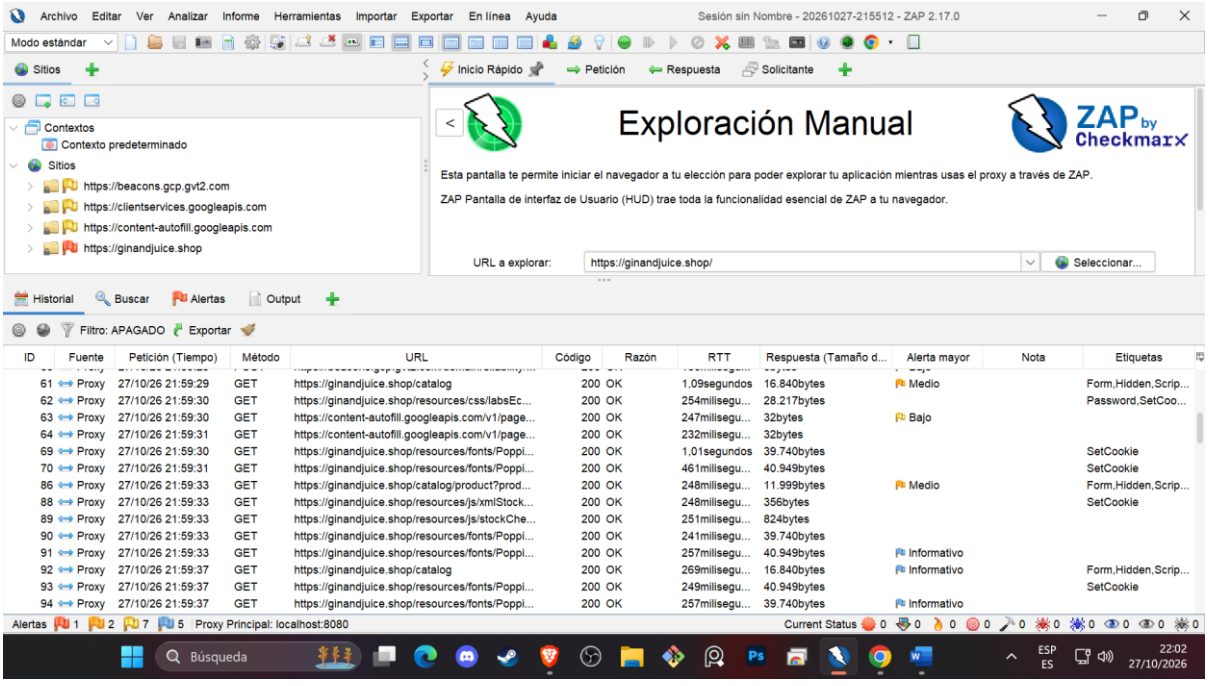
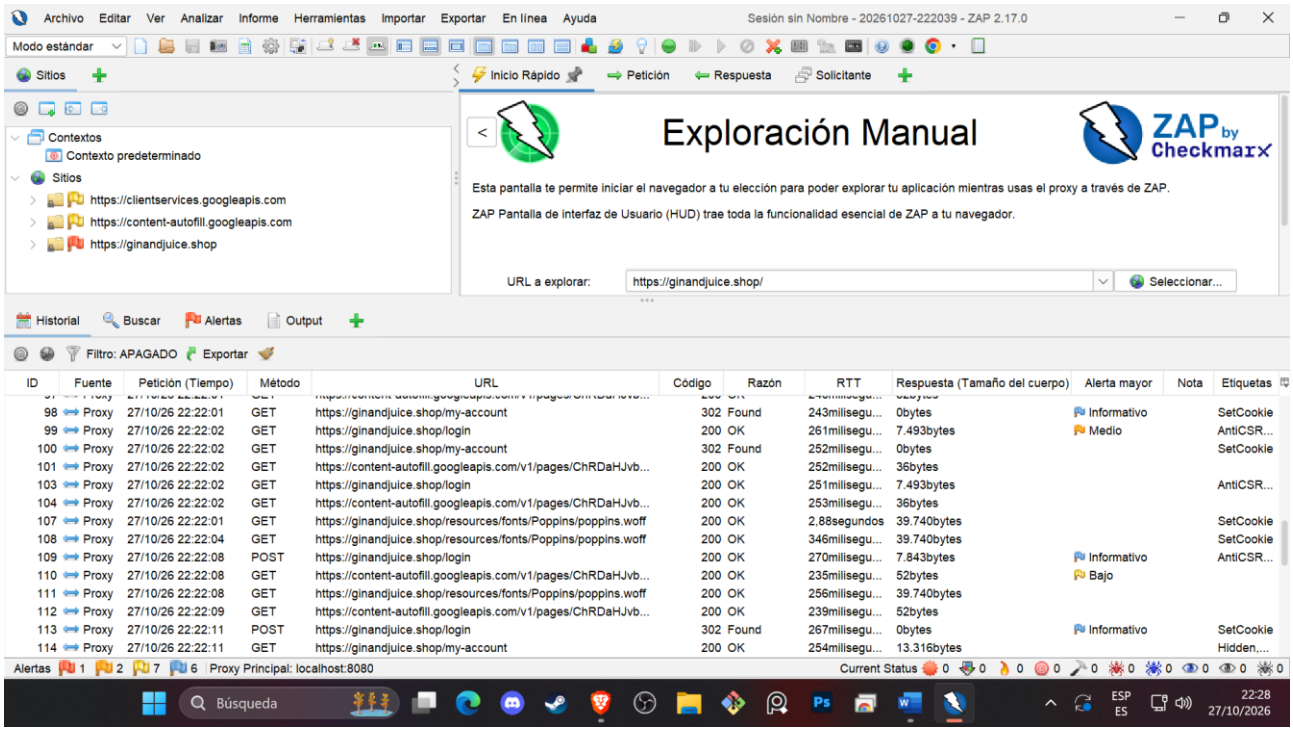



**Imagen 7.** *Tráfico en ZAP al navegar por la sección de productos (/catalog y /catalog/product).*

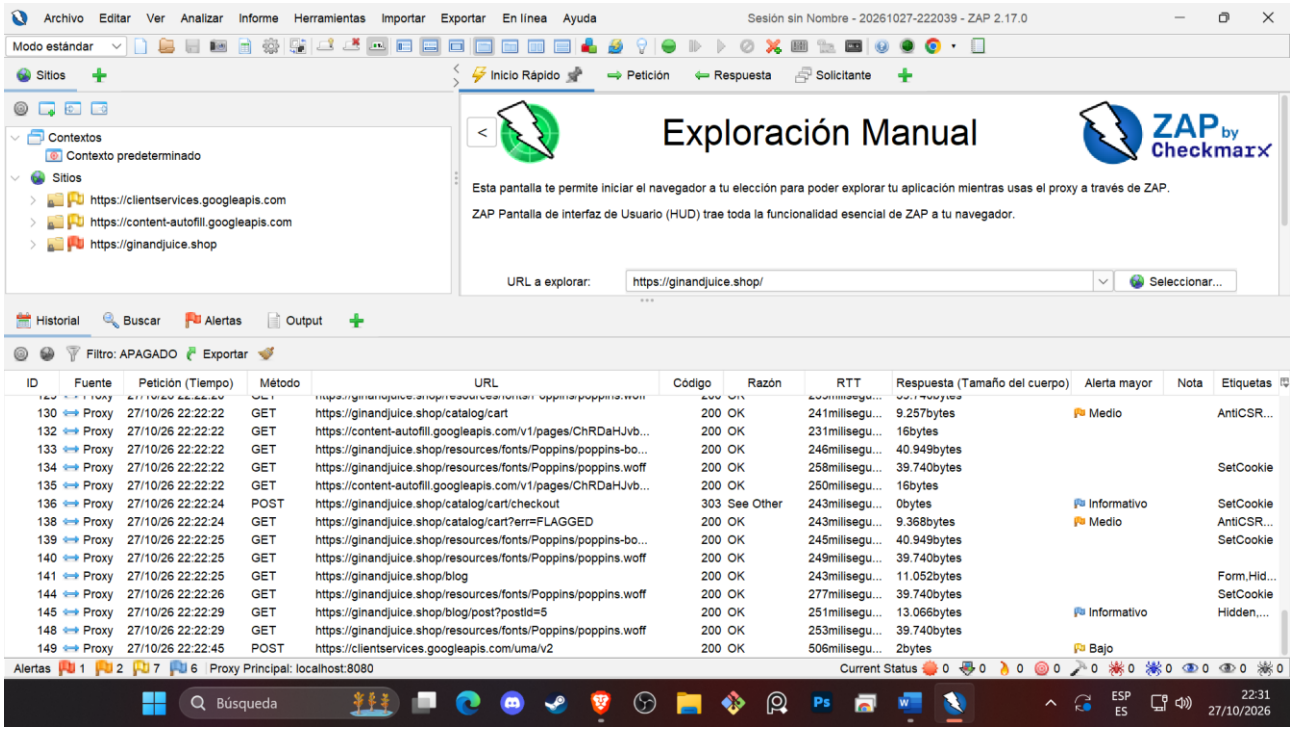
## 2.4 Página de inicio de sesión (Login)




Se realizó la prueba de acceso a la sección de autenticación de la aplicación interactuando con el formulario de inicio de sesión. En la pestaña Historial de OWASP ZAP se registraron las siguientes peticiones clave:

- Acceso inicial: Petición GET [https://ginandjuice.shop/login](https://ginandjuice.shop/login) (ID 99 / ID 103) con código 200 OK, mediante la cual el servidor sirve la interfaz del formulario HTML. ZAP identifica en esta respuesta una alerta de nivel Medio por la falta de un token anti-CSRF en el formulario.
- Intento de autenticación: Solicitud con método POST [https://ginandjuice.shop/login](https://ginandjuice.shop/login) (ID 109 y ID 113). En la petición ID 113, el servidor responde con un código de estado 302 Found (redirección HTTP) y asigna una cookie de sesión (SetCookie).
- Redirección exitosa: Petición posterior GET [https://ginandjuice.shop/my-account](https://ginandjuice.shop/my-account) (ID 114) con respuesta 200 OK, confirmando el acceso al panel o perfil del usuario autenticado.

## 2.5 Carrito de compras




- Se exploró la sección del carrito de compras de la tienda y se simuló el proceso de finalización de compra (checkout). En el historial de ZAP se registraron las siguientes peticiones clave:
- Consulta del carrito: Petición GET [https://ginandjuice.shop/catalog/cart](https://ginandjuice.shop/catalog/cart) (ID 130 y ID 138) con código de respuesta 200 OK. ZAP detecta en esta sección una alerta de nivel Medio etiquetada como AntiCSR (ausencia de protección contra falsificación de peticiones en formularios de acción).
- Proceso de pago / Checkout: Solicitud con método POST [https://ginandjuice.shop/catalog/cart/checkout](https://ginandjuice.shop/catalog/cart/checkout) (ID 136), la cual devuelve un código de estado 303 See Other, indicando la redirección del flujo de la transacción comercial.
- Recursos adicionales: Se observa también la carga paralela de estilos, fuentes y la navegación complementaria hacia la sección del blog de la tienda (/blog, IDs 141 a 148).

## 3. Análisis de las alertas

Con la navegación anterior ZAP generó 15 tipos de alertas distintos: 1 Alta, 2 Medias, 7 Bajas y 5 Informativas (pestaña Alertas). Las marcadas como “(Systemic)” indican que el mismo problema se

repite en muchas URL del sitio; el número entre paréntesis es la cantidad de instancias encontradas. Elegí tres de niveles diferentes (marcadas en rosa en la captura).

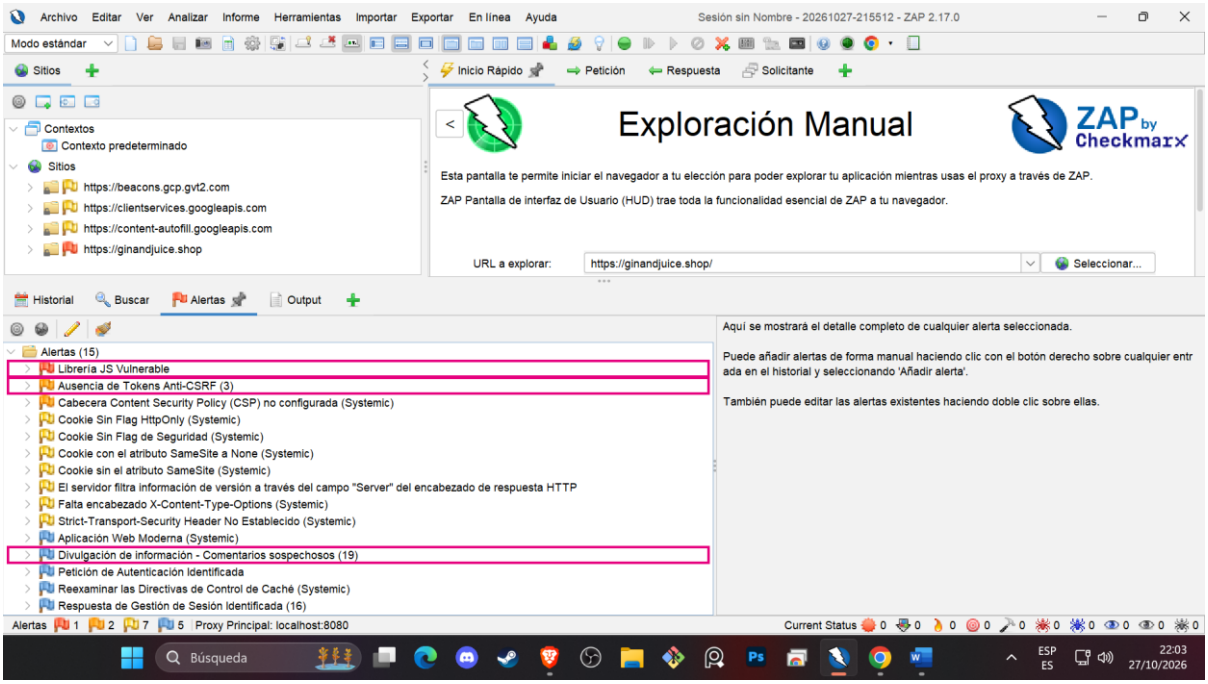



**Figura 8.** *Pestaña Alertas de ZAP: 15 alertas. Resaltadas, las tres analizadas a continuación.*

**Alerta 1: Librería JS Vulnerable (nivel Alto)**

| | |
|---|---|
| **Qué detectó** | ZAP encontró que la aplicación carga una biblioteca JavaScript de terceros con vulnerabilidades conocidas. En el Historial, el único recurso con alerta Alta es el script angular_… (ID 18), por lo que la alerta se relaciona con AngularJS. |
| **Por qué importa** | Usar una versión con fallas públicas le permite a un atacante aprovecharlas sin descubrir nada nuevo. Como el código se ejecuta en el navegador de cada usuario, una falla puede derivar, por ejemplo, en XSS. |
| **Cómo corregirlo** | Actualizar a una versión soportada y sin vulnerabilidades conocidas, o migrar si la librería ya no tiene mantenimiento. |

**Alerta 2: Ausencia de Tokens Anti-CSRF (nivel Medio, 3 instancias)**

| | |
|---|---|
| **Qué detectó** | ZAP encontró 3 formularios HTML que no incluyen un token anti-CSRF. |
| **Por qué importa** | Sin ese token, un sitio malicioso podría lograr que el navegador de un usuario con sesión iniciada envíe una petición que él no quiso hacer (cambiar datos, realizar una acción) sin que se dé cuenta. |

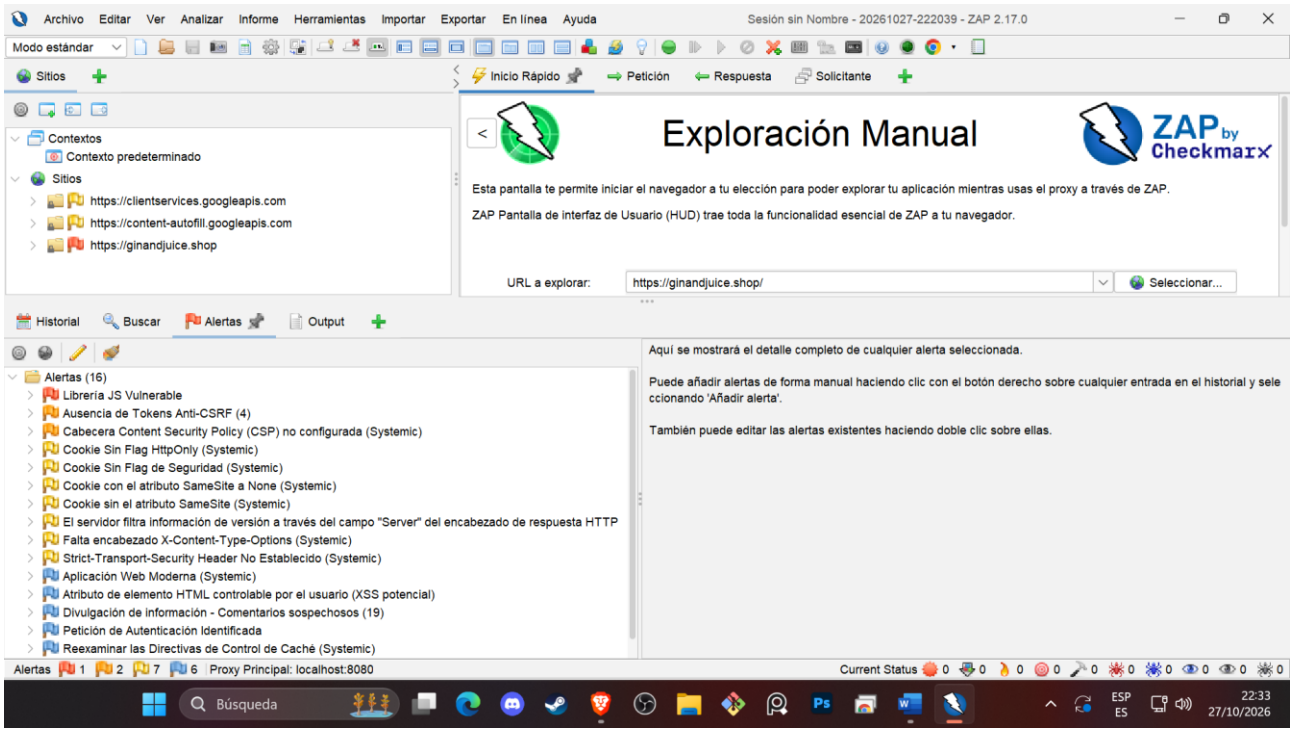
| | |
|---|---|
| **Cómo corregirlo** | Agregar tokens únicos e impredecibles por sesión o formulario y validarlos en el servidor; complementar con cookies SameSite y verificación de los encabezados Origin/Referer. |

**Alerta 3: Divulgación de información, comentarios sospechosos (nivel Informativo, 19 instancias)**

| | |
|---|---|
| **Qué detectó** | ZAP encontró 19 casos de comentarios dentro del código (HTML o JavaScript) con palabras que pueden indicar información interna. |
| **Por qué importa** | Los comentarios olvidados por los desarrolladores pueden revelar rutas, lógica interna o datos de prueba. Por sí solos no son una vulnerabilidad, por eso el nivel es informativo, pero ayudan a quien está haciendo reconocimiento del sitio. |
| **Cómo corregirlo** | Eliminar los comentarios del código que se publica en producción (por ejemplo, con el proceso de build o minificación). |

### 4. Hallazgo principal

Elegí la alerta de mayor nivel, Librería JS Vulnerable, y completé la ficha técnica de 8 elementos.




| | | |
|---|---|---|
| **1** | **Nombre de la alerta** | Librería JS Vulnerable (Vulnerable JS Library) |

| | | |
|---|---|---|
| **2** | **Nivel de riesgo** | Alto (bandera roja en ZAP) |
| **3** | **URL o componente** | https://ginandjuice.shop/resources/js/angular_… (script de AngularJS cargado por la aplicación, ID 18 del Historial). |
| **4** | **Parámetro afectado** | No aplica: es un componente de terceros, no un parámetro de entrada. |
| **5** | **Evidencia** | AngularJS v1.7.7 (según se observa en el campo *Evidencia* del detalle de ZAP para la URL https://ginandjuice.shop/resources/js/angular_1-7-7.js) |
| **6** | **Descripción técnica** | La aplicación sirve una versión de una biblioteca JavaScript que figura en bases de vulnerabilidades públicas; ZAP compara los scripts cargados contra una base de librerías conocidas. Como el script se ejecuta en el navegador de cada visitante, una falla de la librería puede aprovecharse desde ese contexto. |
| **7** | **Impacto potencial** | Según la falla de la versión, un atacante podría ejecutar código JavaScript en el navegador de la víctima (XSS), robar datos de la sesión o alterar lo que ve el usuario (**CWE ID: 1395**). |
| **8** | **Recomendación** | Actualizar a una versión soportada y sin vulnerabilidades conocidas. Si se confirma que es AngularJS, conviene planificar la migración a una alternativa vigente, ya que dejó de recibir soporte oficial a fines de 2021. Sumar al proceso de desarrollo una revisión periódica de dependencias y servir los scripts con una Content Security Policy restrictiva. |

**5. Conclusiones**

**¿Qué aprendí?**

Aprendí a armar un entorno básico de análisis: ZAP como proxy entre el navegador y el sitio, y a leer el Historial y la pestaña de Alertas. Con solo navegar, sin ataques activos, ZAP detectó 15 tipos de alertas. La mayoría son de configuración (cabeceras de seguridad ausentes, cookies sin los flags HttpOnly, Secure o SameSite) y una de nivel Alto por una librería JavaScript vulnerable. También entendí que ZAP registra todo lo que pasa por el proxy, incluso tráfico que no pertenece a la aplicación, y que una alerta pasiva indica un riesgo a revisar, no una vulnerabilidad confirmada.

**¿Qué fue lo más retador?**

Tuve tres dificultades. Primero, separar el tráfico de la aplicación del tráfico que Chrome genera en segundo plano hacia dominios de Google, que aparecen mezclados en el árbol de Sitios; lo resolví concentrándome en el dominio ginandjuice.shop. Segundo, la advertencia “No es seguro” en una dirección HTTPS; entendí que se debe al certificado propio del proxy de ZAP y que es esperable en esta prueba. Tercero, interpretar las alertas: distinguir los niveles de riesgo y las alertas “Systemic” de las puntuales; lo resolví leyendo el detalle de cada alerta antes de clasificarla.

**¿Qué revisaría a continuación?**

En un entorno real, con autorización, revisaría lo siguiente:

- Las secciones que todavía no documenté con tráfico propio: perfil de usuario, búsqueda y blog, además de las pruebas de inicio de sesión y carrito.
- Definir el alcance (contexto) en ZAP para que solo se analice el dominio objetivo.
- Un escaneo activo para comprobar si parámetros como los del catálogo o la búsqueda son vulnerables a inyecciones (SQL, XSS), algo que el análisis pasivo no puede confirmar.
- Una revisión manual de los formularios sin token anti-CSRF.
- La corrección de cabeceras de seguridad (CSP, HSTS, X-Content-Type-Options), de las cookies y de la librería JavaScript, y un nuevo análisis para verificar que las correcciones funcionaron.

**Resumen final del reporte**

Alertas detectadas por ZAP, ordenadas por nivel de riesgo (datos de la pestaña Alertas).

| Nivel de riesgo | Nombre de la alerta | Instancias |
|---|---|---|
| **Alto** | Librería JS Vulnerable | — |
| **Medio** | Ausencia de Tokens Anti-CSRF | 3 |
| **Medio** | Cabecera Content Security Policy (CSP) no configurada | Sistémica |
| **Bajo** | Cookie sin flag HttpOnly | Sistémica |
| **Bajo** | Cookie sin flag de Seguridad (Secure) | Sistémica |
| **Bajo** | Cookie con el atributo SameSite a None | Sistémica |
| **Bajo** | Cookie sin el atributo SameSite | Sistémica |

| Nivel de riesgo | Nombre de la alerta | Instancias |
|---|---|---|
| **Bajo** | El servidor filtra información de versión a través del campo “Server” del encabezado de respuesta HTTP | — |
| **Bajo** | Falta encabezado X-Content-Type-Options | Sistémica |
| **Bajo** | Strict-Transport-Security Header no establecido | Sistémica |
| **Informativo** | Aplicación Web Moderna | Sistémica |
| **Informativo** | Divulgación de información: comentarios sospechosos | 19 |
| **Informativo** | Petición de Autenticación Identificada | — |
| **Informativo** | Reexaminar las Directivas de Control de Caché | Sistémica |
| **Informativo** | Respuesta de Gestión de Sesión Identificada | 16 |